

LAMP: ASP Rule Induction and Optimization with LLMs

David Bunker
University of Potsdam



Headline

LAMP asks an LLM to recover ASP rules from *instance facts* and an *expected answer set*, then checks every candidate with CLINGO. Across 120 synthetic rule-induction benchmarks, the strongest LAMP models nearly matched ILASP, solved every ILASP timeout case, and usually produced smaller, more stratified programs than the synthetic originals.

99.2%
Best final accuracy with LAMP
gpt-5-mini: 119 / 120

98.3%
Best local model result
gpt-oss:20b: 118 / 120

3 / 3
ILASP timeout benchmarks solved
by both strong LAMP models

3–6
Average output literals for correct
solutions vs. 22 or 33 originals

Problem Setting

Writing ASP that is both correct and efficient is hard. LAMP studies a focused version of that challenge: recover a rule set from example facts and the stable model it should produce.

$$AS(F \cup \Pi) = \{A^*\}$$

Here F is the given fact set, A^* is the target answer set, Φ is a bounded hypothesis space, and the goal is to find a small program $\Pi \subseteq \Phi$ that reproduces A^* exactly.

- Both LAMP and ILASP receive the same facts, target answer set, and rule bounds.
- This makes the comparison about *search strategy*: LLM-guided generation versus conflict-driven symbolic search.
- The task is intrinsically difficult: bounded-arity non-ground normal ASP reasoning is Σ_2^P -complete, and learning adds another search layer.

Toy Benchmark

Facts

```
d0(o0).  
d0(o1).  
d1(o1).
```

Target answer set

```
{ d0(o0), d0(o1), d1(o1), d2(o0) }
```

One minimal solution

```
d2(X) :- d0(X), not d1(X).
```

Correctness is judged by the stable model, not by reproducing the exact synthetic source rules. Many different rule sets can yield the same answer set.

Benchmark Design

Parameter	Values used
Predicates	6, 10
Terms	6, 10
Facts	5, 6, 9
Rules	11
Overall literals	2, 3 per rule body
Negative literals	0, 1, 2
Minimum derived atoms	1, 3

- 120 valid synthetic combinations were generated.
- Programs use unary predicates and normal rules only, which keeps the comparison controlled while still requiring nontrivial induction.
- Each benchmark has exactly one answer set and at least one derived atom.

Why This Matters

- ASP is powerful for configuration, planning, and combinatorial reasoning, but writing the rules is often the bottleneck.
- The paper shows that strong current LLMs can already act as useful ASP modeling assistants when a solver is kept in the loop.
- Because the task is rule induction rather than text translation, success here is evidence of structured reasoning rather than prompt memorization.

ILASP Baseline

ILASP solves the same task under the Learning from Answer Sets framework, but does so with conflict-driven symbolic search over an ASP metaprogram rather than iterative LLM generation.

- The hypothesis space is fixed by a mode bias derived from the benchmark profile.
- The target answer set is encoded as a single context-dependent positive example.
- In this study ILASP is the main symbolic baseline, run with a 420s timeout.

LAMP Protocol

- Prompt the LLM with instance facts F , target answer set A^* , rule schema bounds, and a worked example.
- Parse the raw output into an ASP candidate program Π_i .
- Run $F \cup \Pi_i$ with CLINGO.
- If the produced answer set mismatches A^* , feed the mismatch back and ask for a corrected rule set.
- Stop after success or after 3 total attempts.

The key design choice is simple: let the LLM propose rules, but let a solver decide whether those rules are actually correct.

Accuracy Gains From Feedback

System	1st attempt	Final	Failures
<i>gpt-5-mini</i>	109 / 120	119 / 120	1
<i>gpt-oss:20b</i>	100 / 120	118 / 120	2
<i>qwen3-coder:30b</i>	23 / 120	77 / 120	43
ILASP	—	117 / 120	3

Outcome by model: first attempt, feedback success, failed

Feedback recovered 10 extra cases for *gpt-5-mini*, 18 for *gpt-oss:20b*, and 54 for *qwen3-coder:30b*. The loop matters, especially once the first draft is close but not exact.

Runtime And Search Behavior

Observation	Takeaway
ILASP median runtime: 0.66s	Conflict-driven search is usually extremely fast.
ILASP failures: 3, all at 420s	Rare search-space cliffs still matter.
Both strong LAMP models solved all 3 timeout cases	LLM-guided generation explores the space differently.

Program Quality

Mean output literal count $L(\Pi)$ for correct solutions

Original	gpt-5	gpt-oss	qwen	ILASP
22 literals	4.81	5.15	5.85	3.67
33 literals	3.49	3.82	3.84	2.71

Stratified solved programs

System	Stratified / Solved
Original benchmarks	49 / 120
<i>gpt-5-mini</i>	119 / 119
<i>gpt-oss:20b</i>	117 / 118
<i>qwen3-coder:30b</i>	77 / 77
ILASP	117 / 117

Both ILASP and strong LAMP models compressed the synthetic source programs heavily. Correct outputs usually needed only a handful of literals, and almost every solved program became stratified even when the original benchmark was not.

Timeout Cases

Benchmark	gpt-5	gpt-oss	qwen	ILASP
7	✓	✓	×	420s
27	✓	✓	×	420s
67	✓	✓	×	420s

The three ILASP failures were all timeouts, not logical contradictions. Each timeout case had 11 negative literals, and grounded-rule counts between 20 and 27 compared with a dataset average of 13.28. That points to search-space blowup rather than impossibility of the task.

What The Results Suggest

- Strong LLMs can perform nontrivial ASP rule induction even without ASP-specific training.
- A fully local model, *gpt-oss:20b*, is already close to ILASP on this benchmark class.
- ILASP is still the faster symbolic system on average, but LAMP can succeed on some instances where exhaustive symbolic search times out.
- The generated programs are not only correct; they are often cleaner and easier to explain than the synthetic originals.

Limits And Next Steps

- The study uses synthetic benchmarks with unary predicates, normal rules, and exactly one answer set.
- Each model was run once per benchmark at temperature 1, so small score gaps should be interpreted cautiously.
- Richer targets include higher-arity ASP, choice rules, disjunction, aggregates, planning benchmarks, and ASPBench.
- The most interesting extension is a hybrid system: ILASP-style search plus an agentic LLM refinement loop.

Source code, benchmark data, and generated ILASP tasks are available at <https://github.com/dbunker/lamp>.